LangChain & LangGraph

The Complete Training Guide

From Fundamentals to Advanced Architectures

Comprehensive Reference for Developers, Architects & AI Engineers

April 2026

Table of Contents

Part 1: The Problem Landscape — Why LangChain Exists

1.1 The Raw LLM Challenge

Large Language Models like GPT-4, Claude, Gemini, and Llama are extraordinarily powerful at understanding and generating text. However, using them directly in production applications reveals several painful gaps that developers face every single day.

The Core Problems

Problem 1: LLMs are stateless. Every API call to an LLM is independent. The model has no memory of previous conversations. If you want a chatbot that remembers context, you must manually manage conversation history, truncate it when it exceeds token limits, and pass it with every request. This is tedious, error-prone, and varies by provider.

Problem 2: LLMs are isolated from the real world. Out of the box, an LLM cannot browse the web, query your database, read your files, call your APIs, or take action in external systems. It can only process text that you feed it and return text in response. To make it useful in real applications, you need to connect it to external tools and data sources.

Problem 3: Every LLM provider has a different API. OpenAI, Anthropic, Google, Cohere, Hugging Face, and local models all have different request formats, response structures, authentication mechanisms, and capabilities. Switching providers or using multiple models means rewriting significant code.

Problem 4: Complex tasks require multi-step reasoning. Real-world tasks rarely involve a single LLM call. A research assistant might need to search the web, read multiple documents, synthesize information, fact-check claims, and then produce a final report. Orchestrating this manually requires complex control flow, error handling, and state management.

Problem 5: Prompt management becomes a nightmare. As applications grow, you accumulate dozens of prompts. Each needs versioning, testing, and parameterization. Without a structured approach, prompt engineering devolves into scattered string concatenation throughout your codebase.

> **Trainer's Insight**
> Think of LangChain as the "Spring Framework for AI." Just as Spring unified database access, web serving, and dependency injection in Java, LangChain unifies LLM access, tool integration, memory management, and workflow orchestration in the AI world.

1.2 What LangChain Actually Is

LangChain is an open-source framework (Python and JavaScript/TypeScript) that provides a standardized, composable interface for building applications powered by language models. It was created by Harrison Chase in October 2022 and has become the most widely adopted LLM application framework in the world.

At its core, LangChain solves the problems above by providing five foundational capabilities:

1. **Unified Model Interface —** A single abstraction to talk to any LLM, embedding model, or chat model regardless of the provider behind it.
2. **Prompt Templates —** Structured, reusable, parameterized prompt management that separates prompt logic from application logic.
3. **Tool & Retrieval Integration —** A plug-and-play system to connect LLMs with external tools, APIs, databases, vector stores, and document loaders.
4. **Memory Systems —** Built-in mechanisms to give conversations persistence and context awareness across multiple turns.
5. **Composable Chains & Agents —** Building blocks that let you wire together multi-step LLM workflows, from simple sequential chains to autonomous agents.

1.3 The LangChain Ecosystem at a Glance

Component	What It Does	When You Use It
langchain-core	Base abstractions: Runnables, prompt templates, output parsers, chat message types	Always — it's the foundation of everything else
langchain	Chains, agents, retrieval strategies, higher-level orchestration	Building standard LLM workflows and applications
langchain-community	Third-party integrations: vector stores, tools, document loaders, LLM wrappers	Connecting to external services and data sources
LangGraph	Stateful, graph-based agent orchestration with cycles, branching, and human-in-the-loop	Complex, multi-step agent workflows that need fine-grained control
LangSmith	Observability, tracing, evaluation, and monitoring platform	Debugging, testing, and monitoring production LLM apps
LangServe	Deploy LangChain chains as REST APIs instantly	Serving your chains/agents as production APIs

Part 2: LangChain Core Concepts — Deep Dive

2.1 The Runnable Protocol (LCEL)

The LangChain Expression Language (LCEL) is the heart of modern LangChain. Every component in LangChain implements the Runnable interface, which means every component can be invoked, streamed, batched, and composed using a consistent API.

Why LCEL Matters

Before LCEL, LangChain used rigid Chain classes that were hard to customize. LCEL replaced them with a pipe-based composition model inspired by Unix pipelines. The key insight is that every step in an LLM workflow transforms an input into an output — just like Unix commands.

The Runnable Interface

Every Runnable exposes these methods:

Method	Description
.invoke(input)	Run the chain on a single input and return the output
.stream(input)	Stream the output token by token (for real-time UIs)
.batch([inputs])	Run on multiple inputs in parallel for throughput
.ainvoke(input)	Async version of invoke (for async Python/JS apps)
.astream(input)	Async streaming for non-blocking real-time output

Composing with the Pipe Operator

The pipe operator (| in Python, .pipe() in JS) connects Runnables sequentially. The output of each step flows as input to the next:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Each component is a Runnable
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple terms."
)
model = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()
```

```
# Compose them with pipes
chain = prompt | model | parser

# Use it
result = chain.invoke({"topic": "quantum computing"})
```

Trainer's Insight

Think of LCEL like building with LEGO blocks. Each Runnable is a block with standardized connectors. You snap them together in any order. Need to add logging? Snap in a RunnableLambda. Need parallel paths? Use RunnableParallel. The interface is always the same.

2.2 Models — The Unified Interface

LangChain provides three model abstractions that normalize the differences between providers:

LLMs (Completion Models)

These take a string and return a string. Think of the older GPT-3 text-davinci style models. While less common now, they are still supported for backward compatibility and for providers that offer completion endpoints.

Chat Models

The primary abstraction used today. Chat models take a list of messages (System, Human, AI, Tool) and return an AI message. Every major provider is supported through dedicated packages:

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

# All share the same interface!
openai_model = ChatOpenAI(model="gpt-4o")
claude_model = ChatAnthropic(model="claude-sonnet-4-20250514")
gemini_model = ChatGoogleGenerativeAI(model="gemini-1.5-pro")

# Swap models without changing any other code
chain = prompt | openai_model | parser
chain = prompt | claude_model | parser   # Just change this line
```

Embedding Models

These convert text into numerical vectors for similarity search, clustering, and retrieval. Used primarily in RAG (Retrieval-Augmented Generation) pipelines to embed documents and queries into the same vector space.

```
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector = embeddings.embed_query("What is LangChain?")
```

2.3 Prompt Templates

Prompt templates separate the structure of a prompt from the dynamic data that fills it. This is critical for maintainability, testing, and reuse.

Types of Prompt Templates

Template Type	Use Case	Key Feature
PromptTemplate	Simple string formatting with variables	Basic {variable} substitution
ChatPromptTemplate	Multi-message conversations with roles	System, Human, AI message slots
FewShotPromptTemplate	Include examples to guide the model	Dynamic example selection
MessagesPlaceholder	Inject conversation history dynamically	Memory integration

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role} assistant. Be {tone}."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}"),
])

# Fill in the variables
messages = prompt.invoke({
    "role": "coding",
    "tone": "concise",
    "history": [],
    "question": "How do I sort a list in Python?"
})
```

2.4 Output Parsers

Output parsers transform the raw text output of an LLM into structured data that your application can work with programmatically. This bridges the gap between the unstructured world of language and the structured world of software.

Parser	Output Type	Best For
StrOutputParser	Plain string	Simple text responses
JsonOutputParser	Python dict / JSON object	API-ready structured data
PydanticOutputParser	Validated Pydantic model instance	Type-safe, schema-validated data
CommaSeparatedListOutputParser	List of strings	Generating lists of items
XMLOutputParser	Dict parsed from XML	When XML is more reliable than JSON

2.5 Document Loaders & Text Splitters

In real-world applications, you need to feed external data into your LLM. Document loaders ingest data from virtually any source, while text splitters break large documents into chunks that fit within LLM context windows.

Document Loaders

LangChain provides 160+ document loaders. Some of the most used include:

- **PyPDFLoader —** Loads and extracts text from PDF files, page by page.
- **CSVLoader —** Converts each row of a CSV into a Document object.
- **WebBaseLoader —** Scrapes and loads content from web URLs.
- **DirectoryLoader —** Recursively loads all files from a directory.
- **UnstructuredLoader —** Handles dozens of formats via the Unstructured library (Word, PowerPoint, HTML, etc.).
- **NotionDBLoader, S3FileLoader, GitLoader —** Load from cloud services and repositories.

Text Splitters

Once loaded, documents typically need to be split into smaller chunks for embedding and retrieval. The choice of splitter directly impacts retrieval quality.

Splitter	Strategy	Best For

Splitter	Strategy	Best For
RecursiveCharacterTextSplitter	Splits on paragraphs, then sentences, then words	General purpose (recommended default)
TokenTextSplitter	Splits by exact token count	When you need precise token budgets
MarkdownHeaderTextSplitter	Splits at Markdown headers	Documentation and structured Markdown
SemanticChunker	Splits based on embedding similarity	Preserving semantic coherence

Common Mistake

Using chunk sizes that are too small loses context; too large wastes token budget and dilutes relevance. Start with 1000 characters and 200 overlap, then tune based on retrieval quality in your specific use case.

Part 3: Retrieval-Augmented Generation (RAG)

3.1 Why RAG Is Essential

LLMs have a knowledge cutoff and cannot access your private data. RAG solves this by retrieving relevant documents at query time and injecting them into the LLM's context. Instead of fine-tuning an entire model (expensive, slow, and rigid), you give the model the right information at the right time.

The RAG Pipeline

1. **Ingest:** Load your documents using document loaders.
2. **Split:** Break documents into chunks using text splitters.
3. **Embed:** Convert chunks into vectors using an embedding model.
4. **Store:** Save vectors in a vector store for fast similarity search.
5. **Retrieve:** When a user asks a question, embed the query and find the most similar document chunks.
6. **Generate:** Pass the retrieved chunks + the user's question to the LLM for a grounded answer.

3.2 Vector Stores

Vector stores are specialized databases optimized for similarity search over high-dimensional embedding vectors. LangChain integrates with 50+ vector stores:

Vector Store	Type	Strengths	Best For
FAISS	Local / In-memory	Fast, no server needed	Prototyping, small data
ChromaDB	Local / Embedded	Easy setup, metadata filtering	Development, moderate scale
Pinecone	Managed Cloud	Scalable, real-time updates	Production at scale
Weaviate	Self-hosted / Cloud	Hybrid search, GraphQL	Complex retrieval needs
pgvector	PostgreSQL Extension	Use existing Postgres	Teams already on Postgres
Qdrant	Self-hosted / Cloud	Advanced filtering, fast	High-performance search

3.3 Retrieval Strategies

Basic similarity search is just the beginning. LangChain provides advanced retrieval strategies that dramatically improve answer quality:

- **Similarity Search —** The default. Embed the query, find the K nearest vectors. Simple but effective.
- **MMR (Maximum Marginal Relevance) —** Balances relevance with diversity. Prevents returning 5 nearly identical chunks.
- **Self-Query Retrieval —** Uses an LLM to extract structured filters from natural language queries (e.g., “papers about AI from 2024” automatically filters by year=2024).
- **Multi-Query Retrieval —** Generates multiple rephrasings of the query to capture different angles, then merges results.
- **Contextual Compression —** After retrieval, uses an LLM to extract only the relevant portions from each chunk, reducing noise.
- **Parent Document Retrieval —** Embeds small chunks for precise matching, but returns the full parent document for complete context.
- **Ensemble Retrieval —** Combines multiple retrievers (e.g., keyword + semantic) using Reciprocal Rank Fusion.

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain.retrievers import MultiQueryRetriever

# Build vector store
vectorstore = FAISS.from_documents(docs, OpenAIEmbeddings())

# Basic retriever
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 5, "fetch_k": 20}
)

# Advanced: Multi-query for better recall
multi_retriever = MultiQueryRetriever.from_llm(
    retriever=retriever,
    llm=ChatOpenAI(model="gpt-4o")
)
```

Part 4: Memory Systems

4.1 Why Memory Matters

Without memory, every interaction with an LLM is a blank slate. The model has no idea what was said 30 seconds ago. Memory gives your AI application the ability to maintain context across turns, sessions, and even long-term knowledge accumulation.

4.2 Memory Types in LangChain

Memory Type	How It Works	Trade-offs
ConversationBufferMemory	Stores the complete conversation history verbatim	Simple but grows unbounded; will eventually exceed token limits
ConversationBufferWindowMemory	Keeps only the last K turns of conversation	Bounded size; loses older context
ConversationSummaryMemory	Uses an LLM to summarize the conversation so far, keeping a running summary	Compact; costs extra LLM calls; may lose details
ConversationSummaryBufferMemory	Keeps recent messages verbatim + a summary of older messages	Best of both worlds; slightly more complex
ConversationEntityMemory	Extracts and tracks entities (people, places, concepts) and their evolving attributes	Great for entity-rich conversations; overhead for entity extraction
VectorStoreRetrieverMemory	Stores each interaction as an embedding; retrieves semantically relevant past interactions	Excellent for long-term memory; retrieval may miss temporally important context

Trainer's Recommendation

For most chatbot applications, start with ConversationSummaryBufferMemory. It provides a good balance of recent detail and compressed history. For applications where long-term user preferences matter (like personal assistants), use VectorStoreRetrieverMemory alongside a buffer.

Part 5: Tools & Agents

5.1 What Are Tools?

Tools are the bridge between the LLM's reasoning ability and the external world. A tool is any function that the LLM can decide to call based on the conversation context. Tools transform the LLM from a text generator into an actor that can take real-world actions.

Built-in & Community Tools

- **Search Tools —** Tavily, SerpAPI, Google Search, DuckDuckGo, Brave Search for web searching.
- **Code Tools —** PythonREPL, Shell for executing code and system commands.
- **Math Tools —** LLM-Math, Wolfram Alpha for mathematical computation.
- **API Tools —** RequestsGet/Post for calling arbitrary HTTP APIs.
- **Database Tools —** SQL Database toolkit for querying relational databases using natural language.
- **File Tools —** ReadFile, WriteFile, ListDirectory for filesystem operations.
- **Browser Tools —** PlayWright and Selenium wrappers for web browsing and interaction.

Creating Custom Tools

You can turn any Python function into a tool with the @tool decorator:

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city. Use this when the user
    asks about weather conditions in a specific location."""
    # Call a real weather API here
    return f"Weather in {city}: 72°F, sunny"

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the internal product database. Use this when the
    user asks about product availability or specifications."""
    # Query your database
    results = db.search(query, limit=limit)
    return format_results(results)
```

5.2 How Agents Work

An agent is an LLM that has been given access to a set of tools and the ability to decide which tools to use, in what order, and with what inputs. Unlike a fixed chain (which always follows the same steps), an agent dynamically reasons about the best course of action.

The Agent Loop (ReAct Pattern)

Most agents follow the ReAct (Reasoning + Acting) pattern:

1. **Observe:** The agent receives the user's query and any previous observations.
2. **Think:** The LLM reasons about what to do next. It considers which tool would help, or whether it has enough information to respond.
3. **Act:** The agent calls a tool with specific inputs.
4. **Observe:** The tool returns a result, which becomes a new observation.
5. **Repeat or Respond:** The agent decides whether to take another action or give a final answer.

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# Define tools
tools = [get_weather, search_database]

# Create agent prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

# Build the agent
llm = ChatOpenAI(model="gpt-4o")
agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Run it
result = agent_executor.invoke({
    "input": "What's the weather in Tokyo and do we have umbrellas?",
    "chat_history": []
})
```

Important Limitation

The classic AgentExecutor has significant limitations: it only supports linear tool-calling loops, has no built-in support for branching, parallel tool calls, human approval steps, or error recovery. This is exactly the problem that LangGraph was created to solve.

Part 6: LangGraph — The Agent Orchestration Framework

6.1 Why LangGraph Was Created

As developers built increasingly sophisticated AI agents with LangChain, they hit a wall. The sequential chain model and basic AgentExecutor were simply not expressive enough for real-world agent architectures. Consider these scenarios that are impossible or extremely awkward with basic chains:

- **Multi-agent collaboration —** A researcher agent and a writer agent working together, passing findings back and forth.
- **Human-in-the-loop —** An agent that pauses to get human approval before executing a dangerous action (like sending an email or making a purchase).
- **Conditional branching —** Different workflows based on the user's intent (e.g., "if the query is about billing, route to the billing agent; if it's technical, route to support").
- **Cycles and loops —** An agent that iteratively refines its output, checking quality and retrying until it meets a threshold.
- **Error recovery —** If a tool fails, the agent backtracks and tries an alternative approach.
- **Parallel execution —** Calling multiple tools simultaneously and combining their results.
- **Persistent state —** Long-running workflows that survive server restarts and can be resumed later.

LangGraph solves all of these by modeling agent workflows as directed graphs with state, cycles, and checkpoints.

6.2 Core Concepts of LangGraph

Concept 1: StateGraph

A StateGraph is the container for your entire workflow. It defines the shape of the state that flows through the graph and holds all the nodes and edges. Unlike a chain (which is linear), a graph can have branches, loops, and parallel paths.

```
from langgraph.graph import StateGraph
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

# Define the state schema
```

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]  # Chat history
    current_plan: str                        # The agent's plan
    documents: list[str]                     # Retrieved docs
    iteration_count: int                     # Loop counter

# Create the graph
graph = StateGraph(AgentState)
```

Concept 2: Nodes

Nodes are the processing units of your graph. Each node is a Python function that takes the current state as input and returns updates to the state. Nodes can call LLMs, execute tools, perform calculations, or any other logic.

```
def research_node(state: AgentState) -> dict:
    """Retrieve relevant documents based on the query."""
    query = state["messages"][-1].content
    docs = retriever.invoke(query)
    return {"documents": [d.page_content for d in docs]}

def generate_node(state: AgentState) -> dict:
    """Generate a response using retrieved documents."""
    context = "\n".join(state["documents"])
    prompt = f"Based on: {context}\nAnswer: {state['messages'][-1].content}"
    response = llm.invoke(prompt)
    return {"messages": [response]}

# Add nodes to graph
graph.add_node("research", research_node)
graph.add_node("generate", generate_node)
```

Concept 3: Edges (Control Flow)

Edges define how control flows between nodes. LangGraph supports three types:

Edge Type	Behavior	Use Case
Normal Edge	Always goes from node A to node B	Sequential steps that always follow each other
Conditional Edge	A router function decides which node to go to next, based on the current state	Intent routing, quality checks, branching logic
Entry Edge	Defines which node starts the graph execution	Always needed: sets the entry point

```
# Conditional edge: route based on quality check
def should_retry(state: AgentState) -> str:
    """Decide whether the answer is good enough."""
    if state["iteration_count"] >= 3:
        return "finalize"        # Stop after 3 tries
    if quality_check(state):
        return "finalize"        # Answer is good
    return "research"            # Try again

graph.add_conditional_edges(
    "generate",                  # From this node...
    should_retry,                # Use this function to decide...
    {
        "research": "research",  # If "research" -> go to research node
        "finalize": "finalize",  # If "finalize" -> go to finalize node
    }
)
```

Concept 4: Checkpointing & Persistence

One of LangGraph's most powerful features is built-in checkpointing. After every node execution, the entire state is saved to a checkpoint store. This enables:

- **Time-travel debugging —** Inspect the state at any point in the workflow's history.
- **Resumability —** If the server crashes, the workflow resumes from its last checkpoint.
- **Human-in-the-loop —** Pause the graph, wait for human input (even hours later), then resume exactly where it left off.
- **Branching exploration —** Fork from any checkpoint to explore alternative paths.

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.postgres import PostgresSaver

# In-memory for development
checkpointer = MemorySaver()

# PostgreSQL for production
checkpointer = PostgresSaver.from_conn_string("postgresql://...")

# Compile graph with checkpointing
app = graph.compile(checkpointer=checkpointer)

# Each conversation gets a unique thread
config = {"configurable": {"thread_id": "user-123-session-456"}}
result = app.invoke(initial_state, config=config)
```

Concept 5: Human-in-the-Loop

LangGraph provides first-class support for pausing execution and waiting for human input. You can define interrupt points where the graph stops and waits — the user can then approve, reject, or modify the proposed action before the graph continues.

```
from langgraph.prebuilt import ToolNode

# Compile with interrupt_before to pause before tool execution
app = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["tool_node"]  # Pause before running tools
)

# First invocation: runs until it hits the tool node, then pauses
result = app.invoke({"messages": [("user", "Send email to boss")]}, config)
# >> Agent wants to call: send_email(to="boss@co.com", body="...")
# >> Paused. Waiting for human approval.

# Human approves -> resume
result = app.invoke(None, config)  # Continues from checkpoint
```

6.3 LangGraph Architecture Patterns

Pattern 1: Simple ReAct Agent

The most basic LangGraph pattern replaces AgentExecutor with a graph that has a reasoning node and a tool-execution node connected in a cycle.

Pattern 2: Multi-Agent Supervisor

A supervisor agent coordinates multiple specialized sub-agents. The supervisor decides which agent to delegate to based on the task. Each sub-agent has its own tools and expertise. The supervisor reviews their output and decides whether to delegate further or return the final answer.

Pattern 3: Plan-and-Execute

Separates planning from execution. A planner node creates a step-by-step plan, then an executor node carries out each step sequentially. After each step, a re-planner evaluates progress and adjusts the remaining plan. This is ideal for complex, multi-step tasks like research reports or code generation.

Pattern 4: Reflection / Self-Critique

The agent generates an output, then a critic node evaluates it against quality criteria. If the output fails the check, it loops back to the generator with feedback. This iterative refinement loop produces significantly higher quality outputs than single-pass generation.

Pattern 5: Map-Reduce

For tasks that can be parallelized: a mapper node breaks the task into subtasks, multiple worker nodes process subtasks in parallel, and a reducer node combines the results. Ideal for processing large datasets, analyzing multiple documents, or any embarrassingly parallel workload.

When to Use Which Pattern

Simple Q&A chatbot: Pattern 1 (ReAct). Customer support with multiple domains: Pattern 2 (Supervisor). Research assistant: Pattern 3 (Plan-and-Execute). Content generation with quality standards: Pattern 4 (Reflection). Analyzing 100 documents: Pattern 5 (Map-Reduce).

Part 7: LangGraph Prebuilt Components

7.1 create_react_agent

LangGraph ships with a prebuilt ReAct agent that handles the common case of an LLM reasoning and calling tools in a loop. This is the recommended starting point for most agent applications:

```
from langgraph.prebuilt import create_react_agent

# One line to create a fully functional agent
agent = create_react_agent(
    model=ChatOpenAI(model="gpt-4o"),
    tools=[get_weather, search_database, send_email],
    checkpointer=MemorySaver(),
)

# Use it
result = agent.invoke(
    {"messages": [("user", "Check weather and email me a summary")]},
    config={"configurable": {"thread_id": "session-1"}}
)
```

7.2 ToolNode

The ToolNode is a prebuilt node that automatically executes tool calls from the LLM's response. It inspects the latest AI message for tool_calls, runs the corresponding tools, and returns ToolMessages with the results.

7.3 LangGraph Platform & Cloud

For production deployment, LangGraph offers a platform that includes:

- **LangGraph Server —** A production-ready server that hosts your graph with built-in API endpoints, streaming, and webhook support.
- **LangGraph Studio —** A visual debugger that lets you see the graph structure, inspect state at each node, and step through execution interactively.
- **LangGraph Cloud —** Managed hosting that handles scaling, persistence, and infrastructure so you can focus on your agent logic.
- **Cron Jobs —** Schedule graph executions on a recurring basis.
- **Double-Texting Handling —** Gracefully handles cases where a user sends a new message while the agent is still processing the previous one.

Part 8: LangChain vs. LangGraph — When to Use What

Dimension	LangChain (Chains)	LangGraph
Execution model	Linear pipeline: A → B → C	Directed graph with cycles, branches, parallelism
State management	Passed through the chain; no persistence	Explicit state schema with automatic checkpointing
Human-in-the-loop	Not natively supported	First-class with interrupt_before / interrupt_after
Error recovery	Retry the whole chain or catch exceptions	Checkpoint + resume; conditional retry edges
Multi-agent	Awkward; requires nesting and manual state passing	Natural: each agent is a subgraph
Complexity	Lower; great for simple workflows	Higher initial setup; pays off for complex agents
Best for	RAG pipelines, simple chatbots, linear workflows, prototyping	Autonomous agents, multi-step reasoning, production agent systems

Decision Rule

If your workflow is a straight line (prompt → model → parse), use a LangChain chain. If your workflow has loops, conditions, needs to pause for humans, or involves multiple cooperating agents, use LangGraph. Many production applications use both: LangChain for the components and LangGraph for the orchestration.

Part 9: LangSmith — Observability & Evaluation

9.1 Why Observability Matters

LLM applications are notoriously difficult to debug. When a chain of 5 steps produces a wrong answer, where did it go wrong? Was it the retrieval? The prompt? The model hallucinating? LangSmith provides the answer by capturing detailed traces of every step.

9.2 Key Features

- **Tracing —** Every LLM call, tool invocation, retrieval, and chain step is logged with inputs, outputs, latency, and token counts. You can visualize the entire execution tree.
- **Evaluation —** Build datasets of input/expected-output pairs and run your chains against them. Supports custom evaluators, LLM-as-judge, and exact match metrics.
- **Monitoring —** Production dashboards showing latency, error rates, token usage, and cost over time.
- **Annotation —** Human annotators can label traces as correct/incorrect, building gold datasets for evaluation.
- **Prompt Playground —** Iterate on prompts directly in the LangSmith UI and see results in real time.

```
# Enable LangSmith tracing (just set environment variables)
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your-api-key"
os.environ["LANGCHAIN_PROJECT"] = "my-project"

# Now all LangChain/LangGraph code is automatically traced!
result = chain.invoke({"question": "What is RAG?"})
```

Part 10: Complete End-to-End Example

10.1 Building a Research Assistant with LangGraph

Let's tie everything together with a complete example: a research assistant that can search the web, read documents, synthesize information, and produce a final report — with human approval before publishing.

```
from typing import TypedDict, Annotated, Literal
from langgraph.graph import StateGraph, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode

# --- State Definition ---
class ResearchState(TypedDict):
    messages: Annotated[list, add_messages]
    research_notes: list[str]
    draft_report: str
    approved: bool

# --- Tools ---
@tool
def web_search(query: str) -> str:
    """Search the web for current information."""
    return tavily_client.search(query)

@tool
def read_document(url: str) -> str:
    """Read and extract text from a document URL."""
    return document_loader.load(url)

# --- Nodes ---
def researcher(state):
    """Use tools to gather information."""
    response = llm.bind_tools([web_search, read_document]).invoke(
        state["messages"]
    )
    return {"messages": [response]}

def writer(state):
    """Write a report from research notes."""
    notes = "\n".join(state["research_notes"])
    prompt = f"Write a report based on:\n{notes}"
    report = llm.invoke(prompt)
```

```
    return {"draft_report": report.content}

def publisher(state):
    """Publish the approved report."""
    publish_report(state["draft_report"])
    return {"messages": [("assistant", "Report published!")]}

# --- Routing ---
def route_after_research(state) -> Literal["tools", "writer"]:
    last = state["messages"][-1]
    if last.tool_calls:
        return "tools"
    return "writer"

def route_after_approval(state) -> Literal["publisher", "writer"]:
    if state["approved"]:
        return "publisher"
    return "writer"  # Revise if not approved

# --- Build Graph ---
graph = StateGraph(ResearchState)

graph.add_node("researcher", researcher)
graph.add_node("tools", ToolNode([web_search, read_document]))
graph.add_node("writer", writer)
graph.add_node("publisher", publisher)

graph.set_entry_point("researcher")
graph.add_conditional_edges("researcher", route_after_research)
graph.add_edge("tools", "researcher")
graph.add_edge("writer", END)  # Pauses for human review

app = graph.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["publisher"]  # Human approval required
)
```

Part 11: Best Practices & Production Tips

11.1 Architecture Best Practices

1. **Start simple, add complexity incrementally.** Begin with a basic chain. Add tools only when needed. Move to LangGraph only when chains become insufficient. Premature architecture kills velocity.
2. **Separate concerns.** Keep prompt logic, business logic, and tool logic in separate modules. This makes testing and iteration much easier.
3. **Version your prompts.** Treat prompts like code. Store them in version control. Test them with LangSmith evaluations before deploying changes.
4. **Set token limits and timeouts.** Always set max_tokens on LLM calls and timeouts on tool executions. Unbounded calls will surprise you in production.
5. **Use structured outputs whenever possible.** Function calling and Pydantic output parsers are far more reliable than free-form text parsing.

11.2 RAG Best Practices

1. **Chunk wisely.** Experiment with chunk sizes (500–2000 characters) and overlap (100–300 characters). Evaluate retrieval quality, not just generation quality.
2. **Use hybrid search.** Combine semantic search (embeddings) with keyword search (BM25) for the best retrieval performance.
3. **Add metadata filters.** Tag documents with source, date, category, and use Self-Query to leverage these filters automatically.
4. **Evaluate retrieval separately.** Measure retrieval recall and precision independently from generation quality. Poor retrieval is the #1 cause of bad RAG answers.
5. **Re-rank results.** Use a cross-encoder re-ranker after initial retrieval to boost the most relevant documents to the top.

11.3 Agent Best Practices

1. **Write excellent tool descriptions.** The LLM decides which tool to call based on the tool's description and parameter docs. Poor descriptions lead to wrong tool choices. Be specific, include examples, and state when NOT to use a tool.
2. **Limit the tool set.** More tools means more decision complexity. Group related tools into toolkits and only expose what the agent actually needs.
3. **Add guardrails.** Validate tool inputs before execution. Add confirmation steps for destructive actions. Set iteration limits to prevent infinite loops.

4. **Log everything.** Use LangSmith tracing in production. When an agent misbehaves, you need the full trace to diagnose the issue.
5. **Test with adversarial inputs.** Users will ask your agent to do unexpected things. Build a test suite of edge cases and adversarial prompts.

Part 12: Glossary of Key Terms

Term	Definition
Agent	An LLM that dynamically decides which tools to call and in what order to fulfill a request
Chain	A fixed sequence of processing steps (prompt → model → parser) composed together
Checkpoint	A saved snapshot of the graph state at a point in execution, enabling resume and time-travel
Embedding	A numerical vector representation of text, used for similarity search
LCEL	LangChain Expression Language — the pipe-based composition system for building chains
Node	A processing unit in a LangGraph graph; a function that reads and updates state
RAG	Retrieval-Augmented Generation — retrieving relevant docs to ground LLM responses in facts
ReAct	Reasoning + Acting — the pattern where an agent thinks, acts, observes, and repeats
Runnable	The base interface in LangChain; any component that can be invoked, streamed, or batched
StateGraph	The LangGraph container that defines state schema, nodes, and edges for an agent workflow
Tool	A function that an LLM can call to interact with external systems or perform actions
Vector Store	A database optimized for fast similarity search over embedding vectors

End of Training Guide — LangChain & LangGraph Comprehensive Reference